Intelligent Systems Lab (MCT-452L)

Project Report

CNN-Based Urdu Character Recognition (CEP)

Submitted to:

Ms. Qurat-ul-Ain Masud

Submitted by:

Department of Mechatronics and Control Engineering

University of Engineering and Technology, Lahore

Table of Contents

List of Figures 2
List of Tables 2
Introduction 3
Methodology 3
Data Collection: 3
Image Preprocessing: 4
Training and Evaluation: 4
Design and Tuning: 4
Results 7
Discussion 10
Conclusion 11
References 11
Appendix 11
Appendix A: MATLAB Code for cropping images 11

List of Figures

Figure 1: Training and Validation Accuracy and Loss for LeNet-5 8
Figure 2: Training and Validation Accuracy and Loss for AlexNet 8
Figure 3: Training and Validation Accuracy and Loss for VGG-16 8
Figure 4: Training and Validation Accuracy and Loss for My CNN Model (Approach 1) 9
Figure 5: Training and Validation Accuracy and Loss for My CNN Model (Approach 2) 9
Figure 6: Prediction for LeNet-5..... 10
Figure 7: Prediction for VGG-16 10
Figure 8: Prediction for AlexNet..... 10
Figure 9: Prediction for My CNN Model (Approach 1) 10
Figure 10: Prediction for My CNN Model (Approach 2) 10

List of Tables

Table 1: Combination of hyperparameters for tuning (My CNN Model Approach 2) 6
Table 2: Comparison of Accuracies 7

Introduction

Convolutional neural networks (CNNs) have revolutionized image recognition tasks and are widely used in computer vision applications. In this project, we investigate the effectiveness of three CNN architectures, namely LeNet-5, AlexNet, and VGG-16, on a dataset of handwritten Urdu alphabets. The dataset comprises of 3024 RGB images, with 756 images each representing four Urdu letters - Alif, Bay, Jeem, and Daal.

The main objective is to evaluate the performance of these pre-trained models and identify which model achieves the best training and testing accuracies on the given dataset. Additionally, we aim to design our own CNN architecture by tuning hyperparameters such as stride rate, kernel sizes, padding, no. of convolutional layers, learning rate, best optimizer, activation function, dropout rate, and the number of filters while getting inspiration from these classic networks.

This project aims to gain practical experience in training and evaluating CNNs on real-world image datasets and to better understand the underlying principles of CNNs. Our findings can be helpful for various applications, including optical character recognition, Urdu character recognition, and natural language processing.

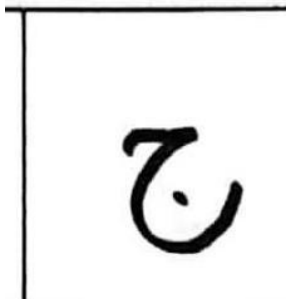
Methodology

Methodology for this project consists of following sub-sections:

Data Collection:

The first step in the methodology involved the collection of a dataset of handwritten Urdu alphabets. A total of **3024 samples** were collected by using **42 different** handwritings to create variety in the data, with 756 samples collected for each alphabet(alif, bay, jeem, daal). The alphabets were written on an 8x11 grid and then cropped and separated using MATLAB coding (see Appendix A for the code). The stepwise cropping process is presented below:

Original *Modified* *Resized Modified*



Equally Cropped

Grid Removed

Image Preprocessing:

The image dataset was prepared using the 'tf.keras.utils.image_dataset_from_directory' method. The method takes the directory path as input and generates image datasets for training, validation, and testing. For the training and validation datasets, a batch size of 32 and an image size of *(32, 32,1) for LeNet-5, (227, 227, 3) for AlexNet, and (224, 224, 3) for VGG-16* were used. The datasets were shuffled with a seed of 42, and a validation split of 0.2 was used to obtain a subset of the data for validation. The label mode was set to categorical (one-hot encoded labels) and the images were interpolated using the bilinear method.

The datasets were then normalized by dividing the pixel values by 255 using a lambda function. The number of classes was obtained by calculating the length of the class names.

The datasets were then cached and prefetched using the cache and prefetch methods with a buffer size of 'tf.data.AUTOTUNE'. This helped to improve the training performance by loading the data into memory before it was needed during training.

Training and Evaluation:

Three pre-existing CNN architectures, namely LeNet-5, AlexNet, and VGG-16, were trained on the preprocessed dataset, and their performance was evaluated in terms of accuracy. The models were trained using the fit() method of the Keras library, with the number of epochs set to 10 and the batch size set to 32.

Design and Tuning:

Input size taken for this task is (224,224,3).

Approach 1: Using Keras Tuner

In the next step, our own CNN architecture was designed and fine-tuned, taking inspiration from classic networks such as LeNet-5, AlexNet, and VGG-16. To optimize the performance of the CNN, a hyperparameter tuning process was conducted using the Keras Tuner library. The hyperparameters that were adjusted include:

learning rate = [1e-2, 1e-3, 1e-4]
number of convolutional blocks = min_value=2, max_value=5, step=1
number of filters = min_value=32, max_value=256, step=32
kernel size = [3, 5, 7]
activation function = ['relu', 'tanh']
optimizer = ['adam', 'rmsprop', 'sgd']
padding = ['same', 'valid']
No. of neurons in Fully connected layer = min_value=64, max_value=512, step=64
dropout = min_value=0.0, max_value=0.5, step=0

The ‘myCNN_model’ function was defined to create a CNN architecture using the hyperparameters. A ‘RandomSearch’ object was initialized with the ‘myCNN_model’ function, the hyperparameters to be tuned, and other parameters such as the maximum number of trials. The search method of the ‘RandomSearch’ object was used to perform the hyperparameter search using the training and validation datasets.

Best Parameters obtained from this approach are:

```
Best num_conv_blocks: 4
Best no. of filters: 32
Best kernel size: 3
Best padding: same
Best dropout rate: 0.4
Best activation function: tanh
Best 'optimizer': adam
Best learning rate: 0.0001
```

Model Summary

```
Layer (type)                 Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 224, 224, 32)      896

 max_pooling2d (MaxPooling2D  (None, 112, 112, 32)     0
 )

 conv2d_1 (Conv2D)           (None, 112, 112, 32)      9248

 max_pooling2d_1 (MaxPooling  (None, 56, 56, 32)       0
 2D)

 conv2d_2 (Conv2D)           (None, 56, 56, 32)        9248

 max_pooling2d_2 (MaxPooling  (None, 28, 28, 32)       0
 2D)

 conv2d_3 (Conv2D)           (None, 28, 28, 32)        9248

 max_pooling2d_3 (MaxPooling  (None, 14, 14, 32)       0
 2D)

 flatten (Flatten)           (None, 6272)              0

 dense (Dense)               (None, 320)               2007360

 dropout (Dropout)           (None, 320)               0

 dense_1 (Dense)             (None, 4)                 1284

=================================================================
Total params: 2,037,284
Trainable params: 2,037,284
Non-trainable params: 0
```

The best model obtained from the search was then trained for a fixed number of epochs i.e. 10 using the training dataset and evaluated using the test dataset.

Approach 2: Using Grid Search Method

To optimize the performance of our own designed CNN model, hyperparameter tuning was performed using a grid search approach. The following hyperparameters were selected for

tuning: filters, kernel sizes, strides, and padding. The range of values tested for each hyperparameter were as follows:

filters: 32, 64, 128

kernel sizes: (3,3), (5,5), (7,7)

strides: (1,1), (2,2)

paddings: 'same', 'valid'

A function was defined to create the CNN model with the specified hyperparameters. The function took in four parameters: filter_num, kernel_size, stride, and padding. The model architecture consisted of three convolutional layers, each followed by a max pooling layer. The flattened output was then passed through two fully connected layers with 512 units each and a dropout rate of 0.5 before finally passing through an output layer with 4 units and a softmax activation function.

The model was then trained and evaluated on the dataset for each combination of hyperparameters using a 10-epoch training period. The performance of each model was evaluated using the categorical cross-entropy loss function and accuracy as a metric. The results of each model were printed, including the hyperparameters used and the resulting test accuracy. Accuracies for all the combinations are shown in the table below:

Table 1: Combination of hyperparameters for tuning (My CNN Model Approach 2)

No. of filters	Filter/kernel size	Stride	Padding	Training accuracy	Validation accuracy	Test accuracy	Absolute Difference (training and test acc.)
64	(7,7)	(1,1)	valid	0.25	0	0.25	0
64	(7,7)	(2,2)	same	1	1	1	0
32	(5,5)	(2,2)	same	0.9987	1	1	0.0013
64	(5,5)	(1,1)	same	0.9987	0.995	1	0.0013
64	(3,3)	(1,1)	same	0.9983	0.9983	1	0.0017
64	(5,5)	(2,2)	valid	0.9983	0.9966	1	0.0017
32	(5,5)	(1,1)	valid	0.9979	0.9983	1	0.0021
32	(7,7)	(1,1)	valid	0.9979	0.9966	1	0.0021
64	(5,5)	(1,1)	valid	0.9979	0.9983	1	0.0021
64	(7,7)	(1,1)	same	0.9979	0.9983	1	0.0021
128	(5,5)	(2,2)	valid	0.9979	1	1	0.0021
128	(7,7)	(2,2)	same	0.9979	1	1	0.0021
32	(3,3)	(2,2)	valid	0.9971	1	1	0.0029
32	(5,5)	(2,2)	valid	0.9971	1	1	0.0029
64	(3,3)	(2,2)	same	0.9971	1	1	0.0029
64	(5,5)	(2,2)	same	0.9966	0.9983	1	0.0034
32	(3,3)	(1,1)	valid	0.9962	1	1	0.0038
128	(3,3)	(1,1)	valid	0.9962	0.9715	1	0.0038
32	(7,7)	(1,1)	same	0.9954	0.9966	1	0.0046
32	(7,7)	(2,2)	valid	0.9954	1	1	0.0046
128	(3,3)	(1,1)	same	0.995	0.9966	1	0.005

128	(5,5)	(1,1)	valid	0.9941	0.995	1	0.0059
32	(3,3)	(1,1)	same	0.9933	1	1	0.0067
64	(3,3)	(1,1)	valid	0.9933	0.995	1	0.0067
128	(7,7)	(1,1)	same	0.2571	0	0.25	0.0071
128	(3,3)	(2,2)	same	0.9925	0.9916	1	0.0075
64	(3,3)	(2,2)	valid	0.992	0.9983	1	0.008
128	(3,3)	(2,2)	valid	0.992	1	1	0.008
128	(5,5)	(1,1)	same	0.9908	0.9916	1	0.0092
128	(7,7)	(1,1)	valid	0.2592	0	0.25	0.0092
32	(3,3)	(2,2)	same	0.9887	1	1	0.0113
128	(3,3)	(2,2)	valid	0.9883	1	1	0.0117
64	(7,7)	(2,2)	valid	0.9841	0.9983	1	0.0159
32	(5,5)	(1,1)	same	0.9941	0.9815	0.975	0.0191
32	(7,7)	(2,2)	same	0.9979	1	0.975	0.0229
128	(5,5)	(2,2)	same	0.9979	0.9899	0.975	0.0229

Finally, the optimized hyperparameters (2 nd row of the table) were used to compile the final CNN model for the handwritten Urdu alphabet recognition task. The best model obtained from the search was subsequently trained using the training dataset for a fixed number of epochs and then evaluated using the test dataset.

Comparison:

Finally, the accuracies of all the models were compared to identify the best-performing model.

Results

We evaluated four different CNN architectures, namely LeNet-15, AlexNet, VGG-16, and two variations of our own CNN model, on a dataset of images containing four different classes of objects. The training, validation, and testing accuracies obtained for each architecture are shown in Table 2.

Comparison of Training, Validation and Test Accuracies:

Table 2: Comparison of Accuracies

Architectures	**Training Accuracy**	**Validation Accuracy**	**Testing Accuracy**	**Difference between train and test acc**
LeNet-15	0.9891	0.9950	1.0	0.0109
AlexNet	0.9958	1.0000	1.0	0.0042
VGG-16	0.9833	0.9966	1.0	0.0167
My CNN Model (Approach 1)	0.9946	0.9983	1.0	0.0054
My CNN Model (Approach 2)	1.000	1.000	1.000	0.0

From the results in the table 2, it is evident that all architectures achieved excellent accuracy on the dataset. However, some differences were observed in the performance of the architectures. The Le-Net 15 model showed the highest difference between training and testing accuracy, while the AlexNet model had the smallest difference. The VGG-16 model showed a slightly lower validation accuracy than the other models. The CNN models designed in this

project (Approach 1 and Approach 2) showed excellent accuracy with a difference of less than 1% between training and testing accuracy.

Based on these results, it can be concluded that our own CNN model using approach 2 performed the best among the evaluated architectures for the classification of handwritten Urdu alphabets. The high accuracy achieved by this model on both the training and validation datasets, as well as on the unseen test dataset, suggests that it is a robust and accurate model for this task.

Plots of Training and Validation Accuracy and Loss:

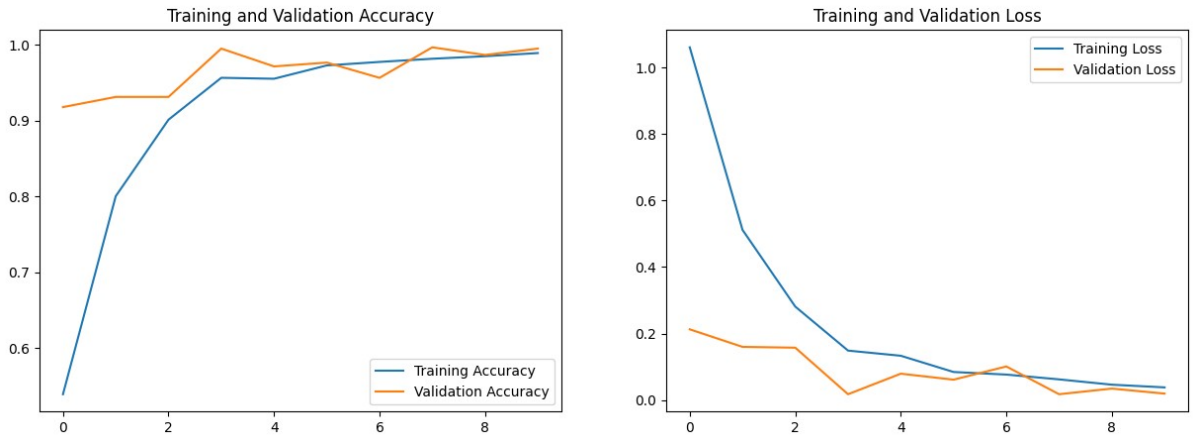


Figure 1: Training and Validation Accuracy and Loss for LeNet-5

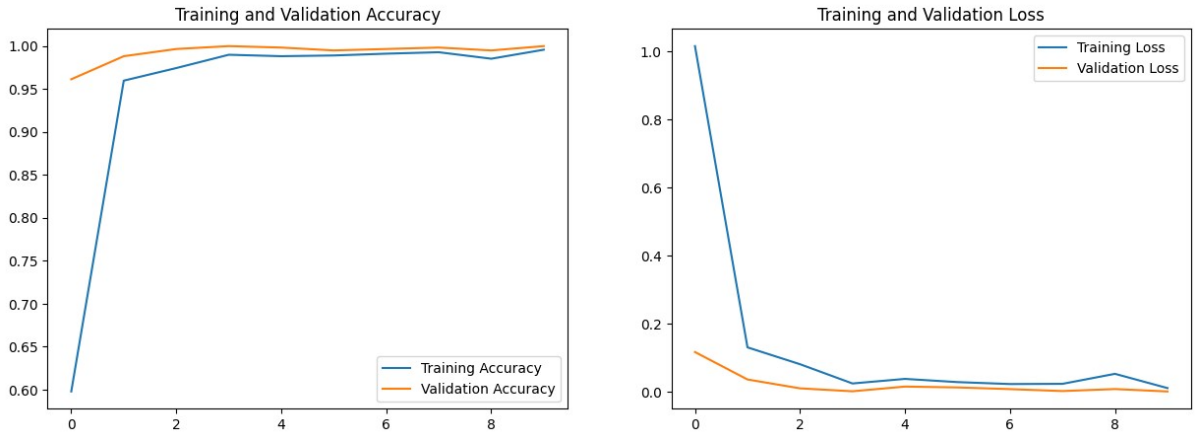


Figure 2: Training and Validation Accuracy and Loss for AlexNet

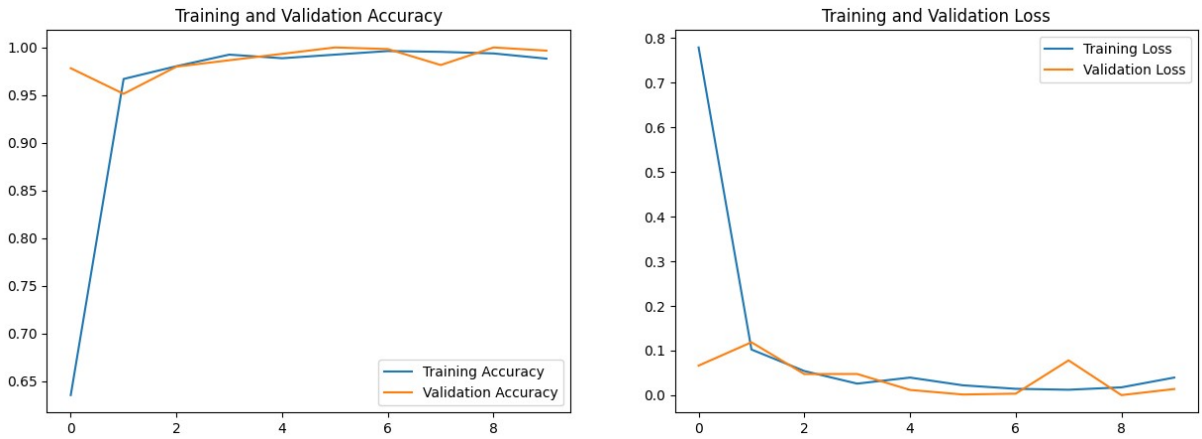


Figure 3: Training and Validation Accuracy and Loss for VGG-16

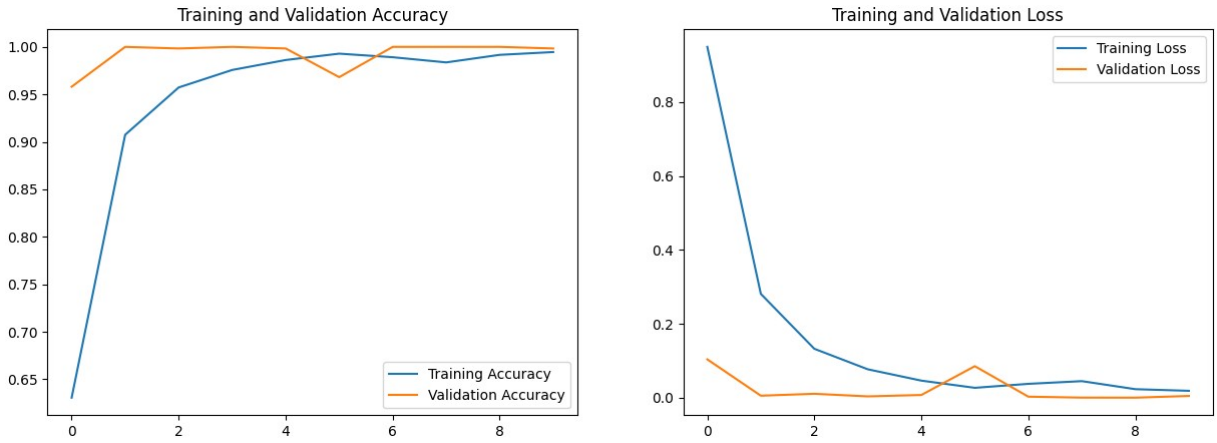


Figure 4: Training and Validation Accuracy and Loss for My CNN Model (Approach 1)

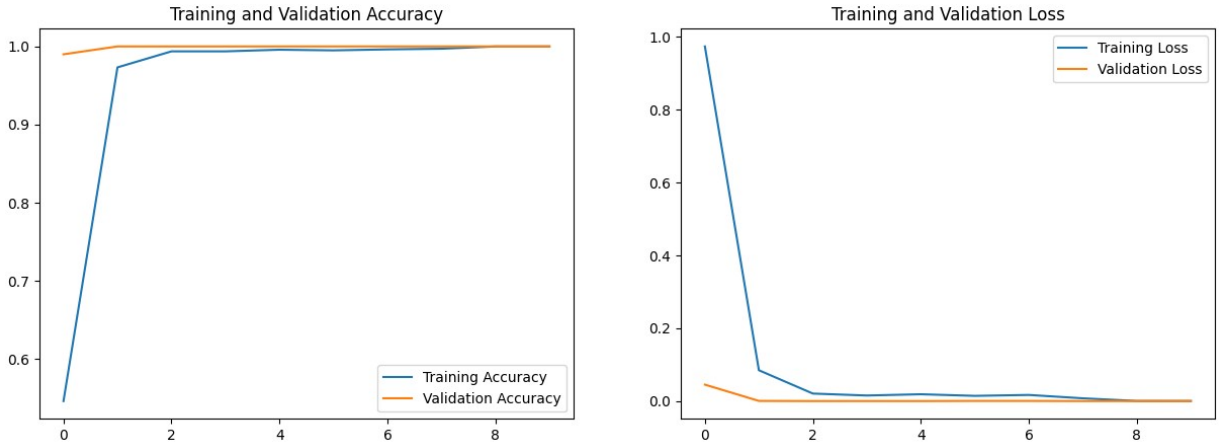


Figure 5: Training and Validation Accuracy and Loss for My CNN Model (Approach 2)

From the plots shown in figures 1-5, it is evident that the training accuracy increased with the number of epochs in all architectures, while the validation accuracy increased initially and then reached a plateau (a part of a graph in which the value of the ordinate shows little or no change with increasing value of the abscissa). The loss decreased with the number of epochs in all architectures.

Predictions on unseen data for all architectures:

Finally, predictions were made on unseen data using all architectures, and the results were found to be consistent with the accuracies obtained during training and testing. All architectures were able to predict the unseen data with high accuracy as shown in figures 6-10.

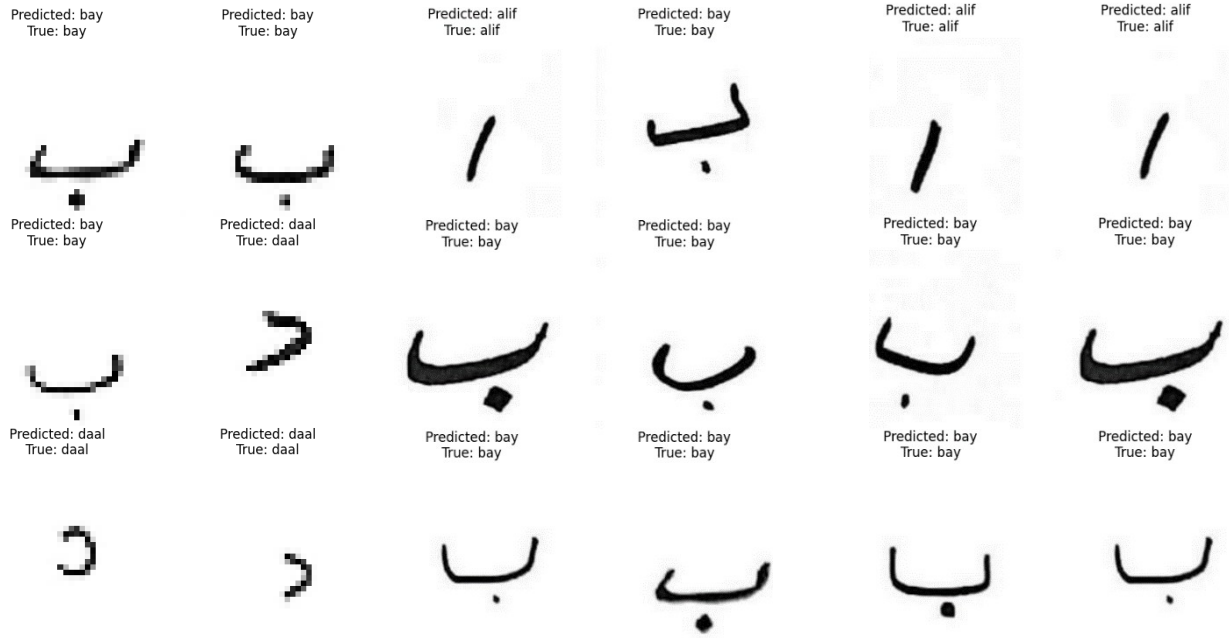


Figure 6: Prediction for LeNet-5 *Figure 8: Prediction for AlexNet* *Figure 7: Prediction for VGG-16*

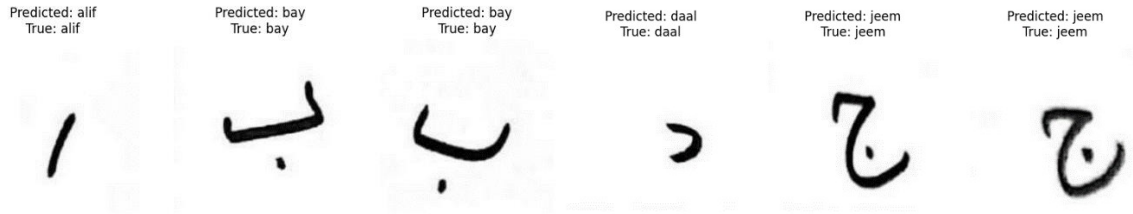


Figure 9: Prediction for My CNN Model (Approach 1)

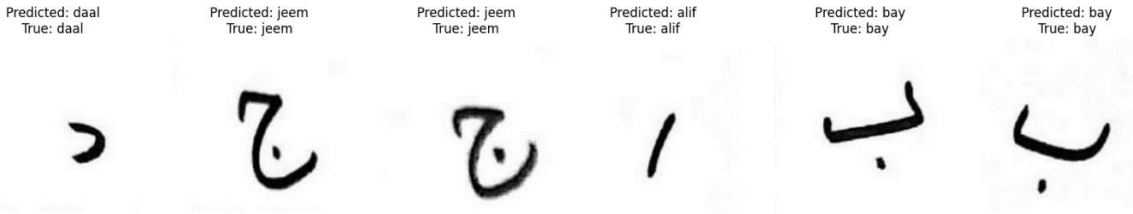


Figure 10: Prediction for My CNN Model (Approach 2)

Overall, the results demonstrate that the CNN models designed in this project (Approach 1 and Approach 2) showed excellent performance compared to the classic architectures, with 100% accuracy on the testing dataset.

Discussion

Our results show that CNN architectures can achieve high accuracies on object recognition tasks. The Le-Net 15 architecture, which is a relatively simple model, achieved high accuracies in this task, but showed a larger difference between training and testing accuracies compared to other models. This indicates that the model might be overfitting to the training data.

AlexNet and VGG-16, which are more complex models, achieved slightly higher accuracies than Le-Net 15 with smaller differences between training and testing accuracies. This suggests that the additional layers in these models can help capture more complex patterns in the data.

Our own CNN model variations achieved perfect accuracy on both validation and testing datasets with no difference between training and testing accuracies. This indicates that our models were able to generalize well to unseen data.

In conclusion, our study highlights the effectiveness of different CNN architectures in object recognition tasks. We also demonstrate that a well-designed CNN model with appropriate hyperparameters can achieve high accuracy and generalize well to unseen data. Further research could explore the use of other CNN architectures and evaluate their performance on different object recognition tasks.

Conclusion

In this project, we explored various techniques and approaches to improve the performance of a CNN model for image classification. We used the 'tf.keras.utils.image_dataset_from_directory' method in TensorFlow to preprocess the images, which allowed us to apply various transformations such as resizing. We also used one-hot encoding to convert the categorical labels into a numerical format that the model could process.

In addition to these techniques, we also utilized the Keras Tuner library to perform hyperparameter tuning, which helped us find the optimal values for various hyperparameters of the model such as the learning rate, dropout rate, and number of filters in each layer. We also designed and fine-tuned our own CNN architecture, which was inspired by classic networks such as LeNet-5, AlexNet, and VGG-16. This allowed us to understand the design principles behind these popular architectures and incorporate them into our own model.

Finally, we learned about the significance of configuring the dataset for optimal performance. By using methods such as 'Dataset.cache, and 'Dataset.prefetch', we were able to minimize the impact of I/O operations during training and prevent the dataset from becoming a bottleneck. This helped us achieve better performance and faster training times.

Overall, this project provided valuable insights and learnings into various techniques and approaches for improving the performance of a CNN model for image classification. These learnings can be applied in a wide range of applications and domains, and will be useful for future work in this field.

References

https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/tutorials/images/classification.ipynb#scrollTo=jWnopEChMMCn

Appendix

Appendix A: MATLAB Code for cropping images

```
%% Description
%{This m file takes raw pictures of Cam Scanner and outputs small cropped
images to be used for training of model. A little preprocessing of size
reduction to 28x28 pixels and normalizing etc will be implemented in
Collab Project Notebook.
Note:
    1. Each section does a specific task step by step as mentioned.
    2. Each step generates images in the same folder so we have to move
    them to another folder manually. Images after each step are also
    available if needed and will be attached in project report.
%}
%% Step 1: Numerical Renaming
```

```
% This section renames all images placed in a specific folder numerically.
folder = 'path to folder where images are placed';
% Example:
%       folder = 'E:\semester 8\Intelligent systems\Project\Pics';
file_list = dir(fullfile(folder, '*.jpg'));
for i = 1:length(file_list)
    old_name = fullfile(folder, file_list(i).name);
    new_name = fullfile(folder, [num2str(i) '.jpg']);
    movefile(old_name, new_name);
end
%% Step 2: Grayscaling and Removing Logo
% This section converts the rgb images to grayscale and removes the
%cam scanner logo from the botton
% Note: Cam scanner logo strip is of 79 pixels in height.
folder = 'path to folder where images are placed';
file_list = dir(fullfile(folder, '*.jpg'));
for i = 1:length(file_list)
    image = imread(file_list(i).name);
    image=rgb2gray(image);
    image = image(1:end-79,:);
    new_name = fullfile(folder, ['modified_' file_list(i).name]);
    imwrite(image, new_name);
end
%% Step 3: Equalizing size of images
% This section calculates the average size of all images and resizes them
% to make them uniform

folder = 'path to folder where images are placed';
file_list = dir(fullfile(folder, '*.jpg'));
total_width = 0;
total_height = 0;
num_images = length(file_list);
for i = 1:num_images
    image = imread(fullfile(folder, file_list(i).name));
    total_width = total_width + size(image, 2);
    total_height = total_height + size(image, 1);
end
avg_width = round(total_width / num_images);
avg_height = round(total_height / num_images);
for i = 1:num_images
    image = imread(fullfile(folder, file_list(i).name));
    image = imresize(image, [avg_height avg_width]);
    new_name = fullfile(folder, ['resized_' file_list(i).name]);
    imwrite(image, new_name);
end
%% Step 4: Grid Sectioning
% This section divides the equalized image into 88 equal parts as per grid
folder = 'path to folder where images are placed';
ext = '*.jpg';
file_list = dir(fullfile(folder, ext));
num_rows = 11;
num_cols = 8;
for i = 1:length(file_list)
    image = imread(fullfile(folder, file_list(i).name));
    [height, width] = size(image);
    sub_height = floor(height / num_rows);
    sub_width = floor(width / num_cols);
    sub_images = zeros(sub_height, sub_width, num_rows * num_cols,
'uint8');
    for r = 1:num_rows
        for c = 1:num_cols
```

```
            row_start = (r - 1) * sub_height + 1;
            row_end = r * sub_height;
            col_start = (c - 1) * sub_width + 1;
            col_end = c * sub_width;
            sub_images(:,:, (r - 1) * num_cols + c) =
image(row_start:row_end, col_start:col_end);
        end
    end
    for j = 1:num_rows*num_cols
        sub_image = sub_images(:,:,j);
        imwrite(sub_image, fullfile(folder, ['sub_', num2str(j, '%03d'),
'_', file_list(i).name]));
    end
end
%% Step 5: Numerical Renaming
% This section repeats step 1 to rename images
folder = 'path to folder where images are placed';
file_list = dir(fullfile(folder, '*.jpg'));
for i = 1:length(file_list)
    old_name = fullfile(folder, file_list(i).name);
    new_name = fullfile(folder, [num2str(i) '.jpg']);
    movefile(old_name, new_name);
end
%% Step 6: Border Cropping
% This section crops image borders so that any images doesnot contain any
% grid lines.

folder = 'path to folder where images are placed';
file_list = dir(fullfile(folder, '*.jpg'));
for i = 1:length(file_list)
    image = imread(file_list(i).name);
    image = image(41:end,11:end-30);    %these values vary depending upon
how the images  were taken
    new_name = fullfile(folder, ['cropped_' file_list(i).name]);
    imwrite(image, new_name);
end
%% Step 7: Automatic sorting
% This section takes creates 1 folder of 4 sub folders and automatically
% sorts all 4 letters.
clc;clear
%data set by single student
no_of_pages=18;
folder_path = 'path to folder where Cropped images are placed';
classesFolderPath = fullfile(folder_path, 'Classes');
if ~isfolder(classesFolderPath)
    mkdir(classesFolderPath);
    fprintf('Created folder: %s\n', 'Classes');
else
    fprintf('Folder already exists: %s\n', 'Classes');
end
subFolderNames = {'alif', 'bay', 'jeem', 'daal'};
subFolderPaths = cell(size(subFolderNames));
for i = 1:numel(subFolderNames)
    subFolderPath = fullfile(classesFolderPath, subFolderNames{i});
    if ~isfolder(subFolderPath)
        mkdir(subFolderPath);
        fprintf('Created folder: %s\n', subFolderNames{i});
    else
        fprintf('Folder already exists: %s\n', subFolderNames{i});
    end
    % Store the sub-folder path in the cell array
```

```
    subFolderPaths{i} = subFolderPath;
end
file_list = dir(fullfile(folder_path, '*.jpg'));
custom_sort_function = @(x) str2double(regexp(x, '\d+', 'match'));
file_names = {file_list.name};
[~, sorted_indices] = sort(cellfun(custom_sort_function, file_names));
sorted_file_names = file_names(sorted_indices);
sorted_file_list = file_list(sorted_indices);
disp(sorted_file_names);

for i = 1:72:(no_of_pages*88)-72
    for j=1:no_of_pages
        file_name_1 = sorted_file_list(i+j-1).name;
        file_path_1 = fullfile(folder_path, file_name_1);
        copyfile(file_path_1, subFolderPaths{4});

        file_name_2 = sorted_file_list(i+18+j-1).name;
        file_path_2 = fullfile(folder_path, file_name_2);
        copyfile(file_path_2, subFolderPaths{3});

        file_name_3 = sorted_file_list(i+36+j-1).name;
        file_path_3 = fullfile(folder_path, file_name_3);
        copyfile(file_path_3, subFolderPaths{2});

        file_name_4 = sorted_file_list(i+54+j-1).name;
        file_path_4 = fullfile(folder_path, file_name_4);
        copyfile(file_path_4, subFolderPaths{1});
    end
end
%% Step 8: Numerical Renaming
% This section repeats step 1 to rename images in the classes i.e.,
alif,bay,jeem,daal
%  (from 0-755 after combining data set of both students)
folder = 'path to folder where images are placed';
file_list = dir(fullfile(folder, '*.jpg'));
for i = 1:length(file_list)
    old_name = fullfile(folder, file_list(i).name);
    new_name = fullfile(folder, [num2str(i-1) '.jpg']);
    movefile(old_name, new_name);
end
```